

Dynamics - Values

Our programs will compute values

$$\text{VAL-NUM} \frac{n \in \mathbb{N}}{\text{num}[n] \text{ val}}$$

$$\text{VAL-STR} \frac{s \in \Sigma^*}{\text{str}[s] \text{ val}}$$

If e val then either $\vdash e : \text{Num}$ or
 $\vdash e : \text{str}$.

Dynamics - Transitions

We are using operational semantics.
(denotational)

Transition system

- closed terms = states
- instruction transitions, which perform computation
- search transition, which determine the evaluation order of the language.

Rules: *instr:*

$$\text{D-PLUS} \frac{n_1 + n_2 = n}{\text{plus}(\text{num}[n_1]; \text{num}[n_2]) \mapsto \text{num}[n]}$$

search:

$$\text{D-PLUS-1} \frac{e_2 \text{ val} \quad e_1 \mapsto e'_1}{\text{plus}(e_1; e_2) \mapsto \text{plus}(e'_1; e_2)}$$

$$\text{D-PLUS-2} \frac{\cancel{e_1 \text{ val}} \quad e_2 \mapsto e'_2}{\text{plus}(e_1; e_2) \mapsto \text{plus}(e_1; e'_2)}$$

$$\text{D-LEN} \frac{|s| = n}{\text{len}(\text{shr}[s]) \mapsto \text{num}[n]}$$

$$\text{D-LEN-1} \frac{e \mapsto e'}{\text{len}(e) \mapsto \text{len}(e')}$$

$$\text{D-LET} \frac{}{\text{let}(e_1; \lambda x. e_2) \mapsto e_2[e_1/x]}$$

$$\begin{array}{l} \text{plus}(\text{len}(\text{shr}[\text{'asdf'}]); \text{num}[4]) \\ \mapsto \# \\ \text{plus}(\text{num}[4]; \text{num}[4]) \end{array}$$

$$\begin{array}{c} | \text{'asdf'} | = 4 \\ \hline \text{len}(\text{shr}[\text{'asdf'}]) \mapsto \text{num}[4] \quad \text{D-LEN} \\ \hline \text{plus}(\text{len}(\text{shr}[\text{'asdf'}]); \text{num}[4]) \mapsto \text{plus}(\text{num}[4]; \text{num}[4]) \quad \text{D-PLUS-1} \end{array}$$

Dynamics - Multi-step transitions

see derivation on prev page

$\text{plus}(\text{len}(\text{stack}()); \text{num}[i])$
 $\mapsto \#$

$\text{plus}(\text{num}[4]; \text{num}[3])$
 $\mapsto \text{D-PLUS}$

$\text{num}[5]$

reflexive transitive closure

every element
is related
to itself

$$x = x$$

(represents taking
no steps)

relations
can be chained
together

(adding one
more step to
our sequence)

$$x = y \quad y = z \Rightarrow x = z$$

$$\mapsto \#$$

D-MULTI-REFL

$$e \mapsto \# e$$

D-MULTI-STEP

$$e \mapsto e' \quad e' \mapsto \# e''$$

$$e \mapsto \# e''$$

$$a \ggg x \ggg \#$$

$\ggg_{\text{REFL}}$

$$e \ggg \# e$$

$$e \ggg e' \quad e' \ggg \# e''$$

$\ggg_{\text{STEP}}$

$$e \ggg \# e''$$

Dynamics - Properties

(Finality) If $e \text{ val}$ then there is no e'
 $e \mapsto e'$.

Proof by Inspection.

(Determinism) If $e \mapsto e_1$ and $e \mapsto e_2$
then $e_1 = e_2$ (up to α equiv)

$\rightarrow_B$
(confluence $\Rightarrow$ determinism)

Notation:

$\underbrace{e \Downarrow v}_{e \text{ evaluates to } v} \stackrel{\text{def}}{=} \underbrace{e \mapsto^* v}_{\text{there is a transition sequence from } e \text{ to } v} \wedge \underbrace{v \text{ val}}_{v \text{ is a value}}$

Due to the determinism of our dynamics v is unique.

Type safety

(Type Safety)

1. (Preservation) If $\vdash e : T$ and $e \mapsto e'$ then $\vdash e' : T$
2. (Progress) If $\vdash e : T$ then either $e \text{ val}$ or $e \mapsto e'$ for some e' .

Präsenation

tomorrow!

Canonical Forms

(Canonical Forms) Suppose e val

1. If $\vdash e:\text{Num}$ then $e = \text{num}[n]$
for some $n \in \mathbb{N}$

2. If $\vdash e:\text{Str}$ then $e = \text{str}[s]$
for some $s \in \Sigma^{\#}$

Proof by inspection.

$\mapsto$